

IME 775 — Lecture 12

KL Divergence, Maximum Likelihood Estimation, and MAP

1. Recap and Roadmap

In Lecture 11 we established:

- **Bayes' theorem** for updating beliefs
- **Entropy** $H(p)$ for quantifying uncertainty
- **Cross-entropy** $H(p, q)$ for measuring encoding cost under a wrong model

Now we complete the toolkit:

- **KL divergence** — the gap between two distributions
- **Conditional entropy** and **mutual information** — uncertainty in one variable given another
- **MLE** — find parameters that maximize data likelihood
- **MAP** — add prior beliefs to MLE

These directly connect to the loss functions and training procedures used in deep learning.

2. KL Divergence (Kullback-Leibler Divergence)

Definition

The KL divergence from distribution p to distribution q :

$$D_{KL}(p||q) = \sum_i p(x_i) \log \frac{p(x_i)}{q(x_i)} = E_p \left[\log \frac{p(X)}{q(X)} \right]$$

For continuous distributions:

$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)} dx$$

Interpretation

$D_{KL}(p||q)$ measures the **extra bits** needed when using code optimized for q to encode samples actually drawn from p .

Equivalently:

$$D_{KL}(p||q) = H(p, q) - H(p)$$

Cross-entropy minus entropy = the "waste" from using the wrong distribution.

Properties

Property	Statement
Non-negativity	$D_{KL}(p q) \geq 0$ always
Zero condition	$D_{KL}(p q) = 0$ iff $p = q$
Not symmetric	$D_{KL}(p q) \neq D_{KL}(q p)$ in general
Not a metric	Violates triangle inequality

Deeper explanation – why asymmetric?

$D_{KL}(p||q)$ weights the log-ratio $\log(p/q)$ by p . So it cares about regions where p places probability mass.

- $D_{KL}(p||q)$: penalizes q wherever p has mass but q does not (if $q(x) \rightarrow 0$ where $p(x) > 0$, divergence $\rightarrow \infty$). This is called the **forward KL** or **moment-matching** direction.
- $D_{KL}(q||p)$: penalizes q wherever q has mass but p does not. This is the **reverse KL** or **mode-seeking** direction.

Practical consequences:

Direction	Behavior	Used in
$D_{KL}(p q)$ (forward)	q must cover all of $p \rightarrow$ spread out	MLE, cross-entropy loss
$D_{KL}(q p)$ (reverse)	q avoids placing mass where p is zero $\rightarrow$ mode-seeking	Variational inference (VAEs)

For a multimodal p , forward KL makes q try to cover all modes (potentially blurry), while reverse KL makes q collapse onto one mode (sharp but incomplete).

Why p weights the log-ratio — unpacking the expectation:

The log-ratio $\log \frac{p(x)}{q(x)}$ measures the pointwise "surprise difference" at each outcome — how much more or less surprised you are under q compared to p . But not all outcomes matter equally. The factor $p(x_i)$ out front comes from the fact that $D_{KL}(p||q)$ is an **expectation under p** : we are averaging this surprise difference over outcomes *as they actually occur when sampling from p* .

This means KL divergence only "sees" regions where p has mass. If $p(x) \approx 0$ somewhere, that term contributes essentially nothing to the sum, regardless of what q does there. Conversely, if $p(x) > 0$ but $q(x) \rightarrow 0$, the log-ratio $\log(p/q) \rightarrow \infty$ and that term is weighted by a nonzero $p(x)$, so the divergence blows up. This is why:

- **Forward KL** $D_{KL}(p||q)$: weighted by p , forces q to cover everywhere p has mass (mode-covering). In deep learning, cross-entropy loss punishes the model heavily for assigning near-zero probability to outcomes that actually appear in the training data — this is exactly forward KL at work.
- **Reverse KL** $D_{KL}(q||p)$: weighted by q , forces q to avoid placing mass where p doesn't (mode-seeking). This is why variational inference (e.g., VAEs) tends to produce sharper but incomplete approximations.

Rule of thumb: whichever distribution is in front (the one being "expected over") controls *which regions matter*. The one inside the log-ratio is the one being penalized for mismatch in those regions.

Moment matching vs. mode seeking — unpacked:

Forward KL — Moment Matching $D_{KL}(p||q)$

The expectation is under p , so every region where $p(x) > 0$ must be accounted for. If q assigns near-zero probability anywhere p has mass, the divergence blows up. The only safe strategy for q is to spread out and cover all of p 's support. When minimized analytically, the optimal q ends up matching the mean, variance, and higher moments of p — hence "moment matching." For a bimodal p , a unimodal Gaussian q will place itself between the two modes, smearing mass across both. It cannot afford to ignore either mode: a blurry but complete approximation.

Reverse KL — Mode Seeking $D_{KL}(q||p)$

Now the expectation is under q . If q places probability somewhere $p \approx 0$, the term $\log(q/p) \rightarrow \infty$ weighted by $q(x) > 0$, blowing up. So q learns to avoid any region where p is small, and retreats to a single high-density region of p (a mode). Other modes are ignored entirely because venturing into low- p territory is catastrophically penalized. The result is a sharp but incomplete approximation — q finds one peak and stays there.

Concrete picture — bimodal p with peaks at $x = -3$ and $x = +3$:

Direction	What q (Gaussian) does
Forward KL	Sits near $x = 0$, covering both peaks even though $x = 0$ has low p -mass — matches the mean Reverse KL Collapses onto one peak (e.g., $x = -3$), ignoring the other entirely

Practical consequences:

Setting	Direction	Consequence
Classifiers / language models (cross-entropy loss)	Forward KL	Model must assign nonzero probability to every observed outcome — no blind spots
VAEs (encoder regularization)	Reverse KL	Latent code concentrates in high-density regions of the

		prior — compact but may miss variation
GANs	Neither directly	Adversarial objective can approximate arbitrary distributions but with training instability

The key tradeoff: if missing a mode is catastrophic (e.g., a language model that refuses to generate certain words), use forward KL. If you want a sharp, precise sample from one region and don't need full coverage, reverse KL is more stable.

Workout: Compute $D_{KL}(p||q)$ for $p = (0.5, 0.5)$ and $q = (0.9, 0.1)$.

Solution:

$$\begin{aligned}
 D_{KL}(p||q) &= 0.5 \log_2 \frac{0.5}{0.9} + 0.5 \log_2 \frac{0.5}{0.1} \\
 &= 0.5 \times (-0.848) + 0.5 \times 2.322 \\
 &= -0.424 + 1.161 = 0.737 \text{ bits}
 \end{aligned}$$

Now the reverse: $D_{KL}(q||p)$:

$$\begin{aligned}
 D_{KL}(q||p) &= 0.9 \log_2 \frac{0.9}{0.5} + 0.1 \log_2 \frac{0.1}{0.5} \\
 &= 0.9 \times 0.848 + 0.1 \times (-2.322) \\
 &= 0.763 - 0.232 = 0.531 \text{ bits}
 \end{aligned}$$

Note: $0.737 \neq 0.531$ — KL divergence is indeed asymmetric.

KL Divergence for Gaussians

For two univariate Gaussians $p = \mathcal{N}(\mu_1, \sigma_1^2)$ and $q = \mathcal{N}(\mu_2, \sigma_2^2)$:

$$D_{KL}(p||q) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

This closed-form is used directly in VAEs, where the encoder output is regularized toward a standard normal $\mathcal{N}(0, 1)$.

Special case – VAE regularizer: If $p = \mathcal{N}(\mu, \sigma^2)$ and $q = \mathcal{N}(0, 1)$:

$$D_{KL}(p||q) = -\frac{1}{2} (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

Important notation note (VAE context):

In this section's notation:

- $p = \mathcal{N}(\mu, \sigma^2)$ is the encoder output distribution for one input (often written as $q_\phi(z | x)$ in VAE papers)
- $q = \mathcal{N}(0, 1)$ is the latent prior (often written as $p(z)$)

So this KL term is "encoder distribution vs. prior," even though the letters p, q may be swapped compared with standard VAE notation.

Worked VAE example (same formula):

Suppose for one input x , the encoder predicts:

- $\mu = 1.0$
- $\log \sigma^2 = -0.693 \Rightarrow \sigma^2 \approx 0.5$

Then:

- $p = \mathcal{N}(1.0, 0.5)$
- $q = \mathcal{N}(0, 1)$

$$D_{KL}(p||q) = -\frac{1}{2} (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

$$= -\frac{1}{2} (1 - 0.693 - 1.0 - 0.5) = -\frac{1}{2} (-1.193) = 0.5965$$

Interpretation: $KL > 0$ because the encoder distribution is shifted ($\mu \neq 0$) and has different spread ($\sigma^2 \neq 1$). Minimizing this term pushes $\mu \rightarrow 0$ and $\sigma^2 \rightarrow 1$.

VAE terms in plain language

- **Latent code:** A compact hidden vector z representing the input. In a VAE, the encoder predicts a distribution over this code (mean and variance), then samples z .
 - **Decoding:** Mapping latent code back to data space, $z \rightarrow \hat{x}$.
 - **Why KL regularization matters:** Without it, latent codes can become arbitrary and overfit training points. KL regularization keeps codes near a shared prior $N(0, 1)$ so the latent space is smooth and sampleable, enabling generation by drawing $z \sim N(0, 1)$ and decoding.
-

3. Conditional Entropy

Definition

The entropy of Y given that we know X :

$$H(Y | X) = - \sum_x \sum_y p(x, y) \log p(y | x) = E_{X,Y}[-\log p(Y | X)]$$

Intuition: Average remaining uncertainty about Y after observing X .

Properties

- $H(Y | X) \leq H(Y)$ – conditioning reduces entropy (or keeps it the same)
- $H(Y | X) = H(Y)$ iff X and Y are independent
- $H(Y | X) = 0$ iff Y is a deterministic function of X

Chain Rule for Entropy

$$H(X, Y) = H(X) + H(Y | X)$$

Joint uncertainty = uncertainty in X + remaining uncertainty in Y after knowing X .

This parallels the product rule: $P(X, Y) = P(X) \cdot P(Y | X)$ – taking logs turns multiplication into addition.

Workout: A dataset has features X (age: young/old) and labels Y (buy: yes/no):

	Buy=Yes	Buy=No
Young	0.15	0.35
Old	0.30	0.20

Compute $H(Y)$ and $H(Y | X)$.

Solution:

$$P(Y=\text{yes}) = 0.15 + 0.30 = 0.45, P(Y=\text{no}) = 0.55$$

$$H(Y) = -(0.45 \log_2 0.45 + 0.55 \log_2 0.55) = -(0.45 \times (-1.152) + 0.55 \times (-0.862)) = 0.993 \text{ bits}$$

For $X=\text{young}$: $P(\text{young}) = 0.50$, $P(\text{yes}|\text{young}) = 0.30$, $P(\text{no}|\text{young}) = 0.70$

How this is computed from the table:

$$P(\text{yes}|\text{young}) = \frac{P(\text{young, yes})}{P(\text{young})} = \frac{0.15}{0.15 + 0.35} = \frac{0.15}{0.50} = 0.30$$

$$H(Y|\text{young}) = -(0.30 \log_2 0.30 + 0.70 \log_2 0.70) = 0.881 \text{ bits}$$

For $X=\text{old}$: $P(\text{old}) = 0.50$, $P(\text{yes}|\text{old}) = 0.60$, $P(\text{no}|\text{old}) = 0.40$

$$H(Y|\text{old}) = -(0.60 \log_2 0.60 + 0.40 \log_2 0.40) = 0.971 \text{ bits}$$

$$H(Y|X) = 0.50 \times 0.881 + 0.50 \times 0.971 = 0.926 \text{ bits}$$

Information gain: $IG = H(Y) - H(Y|X) = 0.993 - 0.926 = 0.067 \text{ bits}$

Knowing age reduces uncertainty about purchasing by 0.067 bits. Decision trees use this metric to choose splits.

4. Mutual Information

Definition

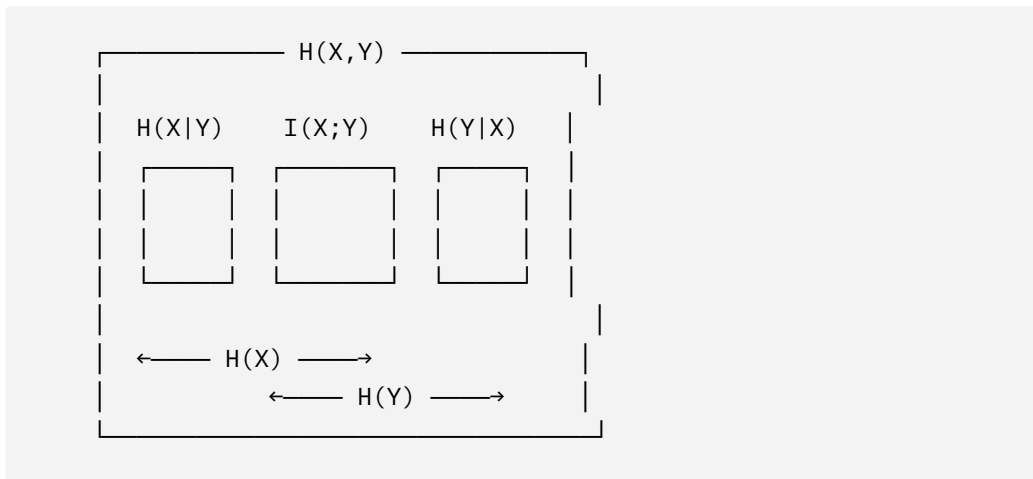
$$I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X) = D_{KL}(p(X, Y) \| p(X)p(Y))$$

Intuition: How much knowing X tells you about Y (and vice versa).

Properties

- $I(X; Y) \geq 0$ always
- $I(X; Y) = 0$ iff $X \perp Y$ (independent)
- $I(X; Y) = I(Y; X)$ (symmetric, unlike KL divergence)
- $I(X; Y) = H(X) + H(Y) - H(X, Y)$

Information Diagram



5. Maximum Likelihood Estimation (MLE)

The Core Idea

Given observed data $\mathcal{D} = \{x_1, x_2, \dots, x_N\}$ and a parametric model $p(x \mid \theta)$, find the parameters that make the observed data most probable:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} P(\mathcal{D} \mid \theta)$$

Likelihood Function

Assuming i.i.d. (independent, identically distributed) data:

$$\mathcal{L}(\theta) = P(\mathcal{D} \mid \theta) = \prod_{i=1}^N p(x_i \mid \theta)$$

Log-Likelihood

Products are numerically unstable (underflow). Take the log:

$$\ell(\theta) = \log \mathcal{L}(\theta) = \sum_{i=1}^N \log p(x_i \mid \theta)$$

Since log is monotonically increasing, maximizing ℓ is equivalent to maximizing $\mathcal{L}$.

MLE for Gaussian

Given $\{x_1, \dots, x_N\}$ from $\mathcal{N}(\mu, \sigma^2)$:

$$\ell(\mu, \sigma^2) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (x_i - \mu)^2$$

Setting $\frac{\partial \ell}{\partial \mu} = 0$ and $\frac{\partial \ell}{\partial \sigma^2} = 0$:

$$\hat{\mu}_{MLE} = \frac{1}{N} \sum_{i=1}^N x_i = \bar{x}$$

$$\hat{\sigma}_{MLE}^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

The MLE for the mean is the sample mean, and for the variance is the (biased) sample variance.

Note: The MLE variance divides by N , not $N - 1$. The unbiased estimator divides by $N - 1$, but MLE is biased for variance. With large N , the difference is negligible.

MLE for Bernoulli

Given $\{x_1, \dots, x_N\}$ where $x_i \in \{0, 1\}$ from $\text{Ber}(\theta)$:

$$\ell(\theta) = \sum_{i=1}^N [x_i \log \theta + (1 - x_i) \log(1 - \theta)]$$

Setting $\frac{d\ell}{d\theta} = 0$:

$$\hat{\theta}_{MLE} = \frac{1}{N} \sum_{i=1}^N x_i = \frac{\text{number of successes}}{N}$$

This is simply the observed proportion — the intuitive estimate.

Workout: You flip a coin 100 times and get 73 heads. What is $\hat{\theta}_{MLE}$?

Solution: $\hat{\theta}_{MLE} = 73/100 = 0.73$

MLE = Minimizing Cross-Entropy

There is a deep connection: maximizing log-likelihood is equivalent to minimizing cross-entropy between the empirical data distribution $\hat{p}$ and the model p_θ :

$$\hat{\theta}_{MLE} = \arg \max_{\theta} \frac{1}{N} \sum_i \log p_\theta(x_i) = \arg \min_{\theta} H(\hat{p}, p_\theta)$$

This means the cross-entropy loss function used in neural networks **is** MLE.

Deeper explanation:

The empirical distribution assigns $\hat{p}(x) = \frac{1}{N} \sum_i \delta(x - x_i)$, placing $1/N$ mass on each data point. The cross-entropy between $\hat{p}$ and the model is:

$$H(\hat{p}, p_\theta) = - \sum_x \hat{p}(x) \log p_\theta(x) = - \frac{1}{N} \sum_{i=1}^N \log p_\theta(x_i)$$

This is exactly the negative average log-likelihood. So minimizing cross-entropy = maximizing likelihood = minimizing KL divergence from data to model (since $H(\hat{p})$ is constant).

Every time you train a neural network with cross-entropy loss, you are doing MLE.

6. Maximum A Posteriori (MAP) Estimation

Motivation: MLE Can Overfit

With limited data, MLE can give extreme estimates. Example: 3 coin flips, all heads $\rightarrow \hat{\theta}_{MLE} = 1.0$. This claims the coin always lands heads, which is almost certainly wrong.

Solution: Incorporate prior beliefs about θ .

MAP Estimation

$$\hat{\theta}_{MAP} = \arg \max_{\theta} P(\theta \mid \mathcal{D}) = \arg \max_{\theta} P(\mathcal{D} \mid \theta) \cdot P(\theta)$$

Taking logs:

$$\hat{\theta}_{MAP} = \arg \max_{\theta} \left[\sum_{i=1}^N \log p(x_i | \theta) + \log p(\theta) \right]$$

The extra term $\log p(\theta)$ acts as a **regularizer** — it penalizes parameter values that are unlikely under the prior.

MAP = Regularized MLE

Prior $P(\theta)$	Regularization Effect
Gaussian: $\theta \sim \mathcal{N}(0, \sigma_0^2)$	$\log p(\theta) \propto -\frac{\ \theta\ ^2}{2\sigma_0^2} \rightarrow$ L2 regularization (weight decay)
Laplace: $\theta \sim \text{Laplace}(0, b)$	$\log p(\theta) \propto -\frac{\ \theta\ _1}{b} \rightarrow$ L1 regularization (sparsity)

This is a fundamental insight: L2 regularization in neural networks is equivalent to MAP estimation with a Gaussian prior on weights. The regularization hyperparameter λ corresponds to the prior variance σ_0^2 :

$$\mathcal{L}_{MAP} = - \sum_i \log p(x_i | \vec{w}) + \frac{\lambda}{2} \|\vec{w}\|^2$$

MAP for Bernoulli with Beta Prior

The Beta distribution $\text{Beta}(\alpha, \beta)$ is the conjugate prior for Bernoulli:

$$p(\theta) = \frac{\theta^{\alpha-1} (1-\theta)^{\beta-1}}{B(\alpha, \beta)}$$

Given k successes in N trials:

$$\hat{\theta}_{MAP} = \frac{k + \alpha - 1}{N + \alpha + \beta - 2}$$

Interpretation: $\alpha - 1$ and $\beta - 1$ act as "pseudo-counts" — virtual observations added before seeing data.

Workout: You flip a coin 3 times, all heads ($k = 3, N = 3$). Compare MLE vs MAP with $\text{Beta}(3, 3)$ prior.

Solution:

MLE: $\hat{\theta} = 3/3 = 1.0$ (extreme — says coin always heads)

MAP: $\hat{\theta} = \frac{3+3-1}{3+3+3-2} = \frac{5}{7} \approx 0.714$

The prior pulls the estimate toward 0.5 (the prior mean of $\text{Beta}(3, 3)$), preventing the extreme MLE answer. With more data, the MAP estimate converges to MLE as the data overwhelms the prior.

7. MLE vs MAP vs Full Bayesian: Summary

Method	Finds	Formula	Analogy
MLE	Point estimate	$\arg \max_{\theta} P(\mathcal{D} \mid \theta)$	Best single guess from data alone
MAP	Point estimate	$\arg \max_{\theta} P(\mathcal{D} \mid \theta) P(\theta)$	Best guess with prior knowledge
Full Bayesian	Entire distribution	$P(\theta \mid \mathcal{D})$	Complete uncertainty quantification

As data increases:

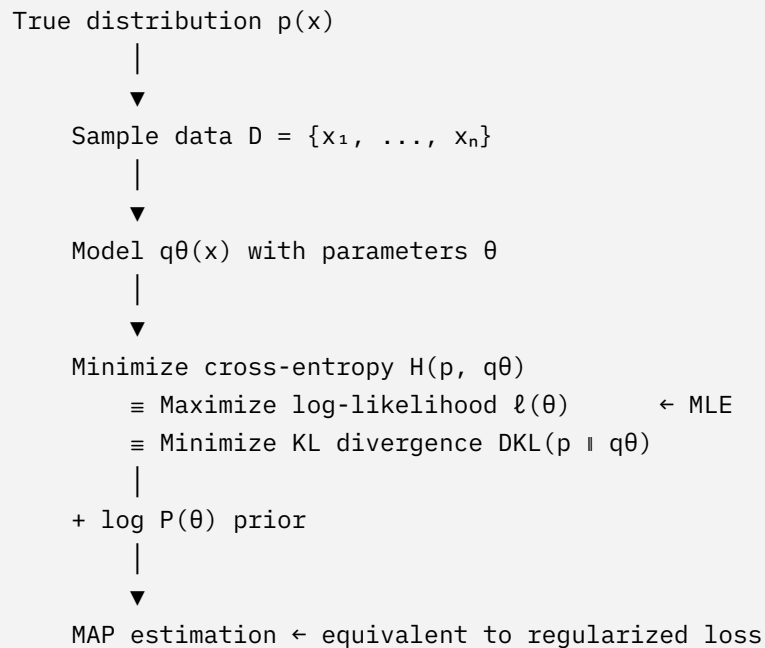
- MLE and MAP converge to the same answer (the prior becomes negligible)
- Full Bayesian posterior concentrates around MLE/MAP (becomes a narrow peak)

When to use which:

- **MLE:** Large datasets, no strong prior information, computational simplicity
- **MAP:** Limited data, known regularization helps, want interpretable regularization

- **Full Bayesian:** Need uncertainty estimates, small data, safety-critical applications
-

8. Putting It All Together: The Loss Function Pipeline



Every component of modern neural network training traces back to these Bayesian and information-theoretic foundations.

Key Takeaways

1. **KL divergence** measures distributional mismatch; it is asymmetric, which has practical consequences for generative models (forward KL → mode-covering, reverse KL → mode-seeking)

2. **Conditional entropy** and **mutual information** quantify how much one variable informs another — the basis of feature selection and information gain in decision trees
 3. **MLE** finds parameters maximizing data likelihood; training with cross-entropy loss **is** MLE
 4. **MAP** adds a prior, producing regularized MLE; Gaussian prior → L2, Laplace prior → L1
 5. The entire loss-function + regularization framework in deep learning is a direct application of Bayesian inference
-

PyTorch Connection

```
import torch
import torch.nn.functional as F
import torch.distributions as D

# KL Divergence between two distributions
p = D.Normal(loc=0.0, scale=1.0)
q = D.Normal(loc=1.0, scale=2.0)
kl = D.kl_divergence(p, q)
print(f"KL(N(0,1) || N(1,2)) = {kl.item():.4f}")

# KL for discrete distributions
p_probs = torch.tensor([0.5, 0.5])
q_probs = torch.tensor([0.9, 0.1])
kl_discrete = F.kl_div(
    q_probs.log(), # KL_div expects log-probabilities for input
    p_probs,       # and probabilities for target
    reduction='sum'
)
print(f"KL discrete = {kl_discrete.item():.4f}")

# MLE for Gaussian: sample mean and variance
data = torch.randn(1000) * 2 + 5 # true: mu=5, sigma=2
mu_mle = data.mean()
var_mle = data.var(correction=0) # MLE uses N, not N-1
```



```

print(f"MLE: mu={mu_mle:.3f}, sigma²={var_mle:.3f}")

# MAP = MLE + regularization
# In PyTorch, L2 regularization (Gaussian prior) via weight_decay:
model = torch.nn.Linear(10, 1)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01,
weight_decay=0.01)
# weight_decay=0.01 is equivalent to MAP with Gaussian prior on
weights

# VAE KL term:  $KL(N(\mu, \sigma^2) || N(0,1))$ 
mu = torch.tensor([0.5, -0.3, 1.2])
log_var = torch.tensor([-0.5, 0.1, -0.2])
kl_vae = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
print(f"VAE KL term: {kl_vae.item():.4f}")

```